



StackMaster Tech

CLOUD · DATA · OBSERVABILITY · SRE

FIELD HANDBOOK · DAY 2 · SPLUNK POWER USER TRACK

Field Extraction, Structured Data & Lookups

A hands-on reference to advanced field creation with **regex**, **erex** & **rex**, self-describing data with **spath** & **multikv**, field aliases and calculated fields, and the complete lookup family — CSV, KV Store, external, geospatial and database.

regex · erex · rex

spath · multikv

Field Aliases

Calculated Fields

Lookups ×5

Same Buttercup Dataset

Vivek Arora

Corporate Trainer, Author & Consultant
StackMaster Tech · Bengaluru, India

youtube.com/@StackMasterTech
github.com/vivek9325
linkedin.com/in/vivek11arora

Contents

ORIENTATION

The Structured-Data Toolkit & Day-2 Dataset	01
---	----

PART A · ADVANCED FIELD CREATION & MANAGEMENT

1 · Regex Foundations in Splunk	02
2 · The erex Command — Extraction by Example	03
3 · The rex Command — Inline Regex Extraction	04
4 · Regex Best Practices	05

PART B · SELF-DESCRIBING DATA & FILES

5 · The spath Command — JSON & XML	06
6 · eval with the spath() Function	07
7 · The multikv Command — Table-Formatted Events	08

PART C · FIELD ALIASES & CALCULATED FIELDS

8 · Field Naming Conventions	09
9 · Creating & Using Field Aliases	10
10 · Creating & Using Calculated Fields	11

PART D · EXPLORING LOOKUPS

11 · Including & Excluding Events with Lookups	12
12 · KV Store Lookups	13
13 · External (Scripted) Lookups	14
14 · Geospatial Lookups	15
15 · Database Lookups (DB Connect)	16
16 · Lookup Best Practices	17

APPENDICES

A · Extraction & Lookup Decision Flowchart	18
B · Day-2 Quick-Reference Cheat Sheet	19

01 · The Structured-Data Toolkit & Day-2 Dataset

Day 1 treated fields as something Splunk hands you. Day 2 is about **making** fields: pulling them out of messy text with regular expressions, reading them straight out of self-describing formats like JSON, and enriching events with data that never appeared in the log at all. Everything here happens at **search time** unless you deliberately push it to index time — which means you can iterate on extractions without re-indexing a single event.

Where each technique fits

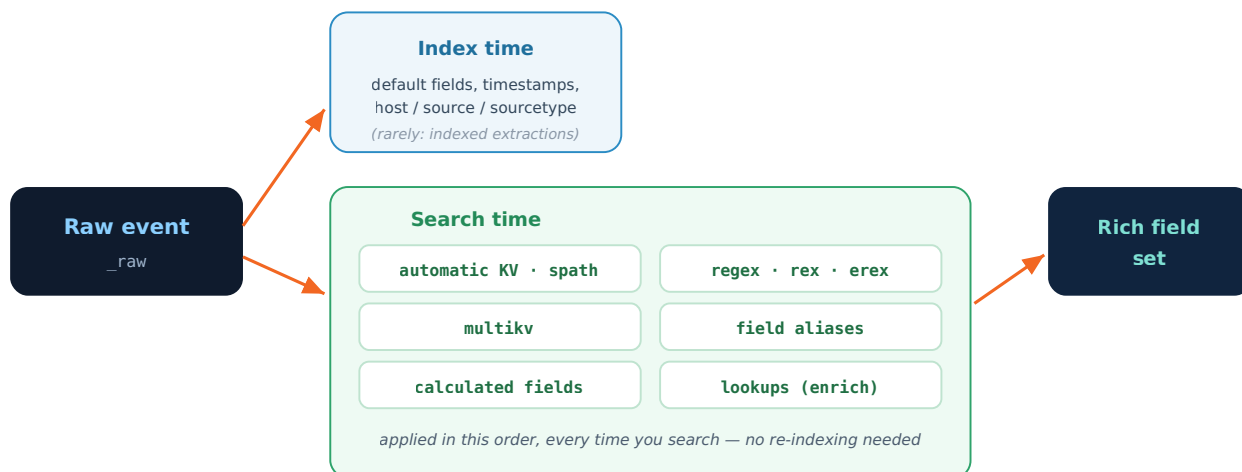


Fig 1.1 — The search-time extraction pipeline. Day 2 lives almost entirely in the green box.

New data sources added for Day 2

The Buttercup Games dataset from Day 1 continues unchanged. Day 2 adds three sources so every technique has real data to bite on:

Source	sourcetype / index	Why it's here
checkout_events.json	checkout_json · web	Nested JSON checkout events (cart items, customer, payment, geo) — for <code>spath</code> and <code>spath()</code> .
proc_report.log	proc_report · web	Events whose body is a fixed-width table (PID/USER/CPU/MEM/CMD) — for <code>multikv</code> .
*_lookup.csv	lookup tables	<code>http_status_lookup</code> , <code>store_geo_lookup</code> , <code>threat_ip_lookup</code> — for the lookup chapters.

LOAD THE ADDITIONS

Upload `checkout_events.json` (sourcetype `checkout_json`, index `web`, set

KV MODE

to auto or run `spath` at search time) and `proc_report.log` (sourcetype `proc_report`, index `web`). Add each CSV under

SETTINGS → LOOKUPS → LOOKUP TABLE FILES

, then define a

LOOKUP DEFINITION

of the same name. All three ship in `Buttercup_Games_Dataset.zip`.

PART A

Advanced Field Creation & Management

When a value is buried in free text, regular expressions are how you get it out. Splunk gives you three on-ramps of increasing control — automatic extraction, **erex** by example, and **rex** by hand — plus a discipline of best practices that keeps those patterns fast and maintainable.

02 · Regex Foundations in Splunk

Splunk uses the **PCRE** (Perl-Compatible Regular Expressions) engine. For field extraction the single most important construct is the **named capture group** — `(?<fieldname>...)` — because the name becomes the field. Everything else is about matching precisely enough to capture the right substring and nothing more.

The building blocks you actually use

Construct	Matches	Example in our data
<code>\d \w \s</code>	digit · word char · whitespace	<code>\d{3}</code> → the <code>200</code> status code
<code>. * + ?</code>	any char · 0+ · 1+ · 0-1	<code>. * ?</code> lazy match up to a delimiter
<code>[...] [^...]</code>	char class · negated class	<code>[^&]+</code> → everything until the next <code>&</code>
<code>(?<name>...)</code>	named capture → field	<code>(?<sku_family>[A-Z]+)</code>
<code>(?:...)</code>	group without capturing	group alternation cheaply
<code>^ \$ \b</code>	start · end · word boundary	anchor to reduce backtracking
<code>(?=...) (?<=...)</code>	look-ahead / look-behind	match position without consuming

Anatomy of an extraction

To pull the SKU family (the letters before the first dash) out of `productId=WC-SH-A01`:

```
# Anchor to the key, capture uppercase letters, stop at the dash
productId=(?<sku_family>[A-Z]+)-
```

- `productId=` — literal anchor so we match the right place, not any capital letters.
- `(?<sku_family>[A-Z]+)` — the named group; `[A-Z]+` is greedy but bounded by the next token.
- `-` — a literal delimiter that terminates the capture cleanly.

TEST BEFORE YOU TRUST

Build patterns against live data with the

FIELD EXTRACTOR

(UI) or a `| rex` in the search bar. Splunk highlights matches instantly, which beats reasoning about regex in your head.

ESCAPE THE LITERALS

Characters `. ? * + ( ) [ ] { } | \ ^ $` are special. To match a literal dot in an IP or filename, write `\.` — a bare `.` matches *anything*.

03 · The erex Command — Extraction by Example

`erex` writes the regex *for* you. You give it a new field name and a few example values you can see in the raw events; Splunk infers a pattern, extracts the field, and — crucially — prints the regex it generated so you can lift it into a `rex` or a permanent extraction.

Syntax

```
| erex <new_field> examples="value1, value2" [counterexamples="..."]
```

Example — pull the session token out of the URL

```
index=web sourcetype=access_combined
| erex session_id examples="SD1SL7FF6ADFF4953, SDT8FI4BBX0FA73B"
| stats dc(session_id) AS unique_sessions
```

Splunk reports the pattern it derived (e.g. `JSESSIONID=(?P<session_id>\w+)`) in the job messages. Copy that into a `rex` for production so the extraction is explicit and fast.

WHEN EREX SHINES

Prototyping and teaching. It is the fastest way to get from "I can see the value" to "I have a field" — and a great regex tutor, because you learn from the patterns it writes.

WHEN TO GRADUATE

`erex` guesses from samples; on messy data it can over- or under-match. Always validate the generated regex on the full set and replace it with a hand-checked `rex` or config-based extraction for anything permanent. Provide `counterexamples` to steer it away from false matches.

04 · The rex Command — Inline Regex Extraction

`rex` is the workhorse. It applies a regex you supply to a field (default `_raw`) and creates one field per named group — entirely at search time, entirely under your control. It also has a second personality: a **sed mode** for search-time substitution and masking.

Two modes

Mode	Syntax	Use
Extraction	<code>rex field=<f> "(?<new>...)"</code>	Create fields from a pattern
Sed (substitute)	<code>rex field=<f> mode=sed "s/.../.../g"</code>	Rewrite / mask values in place

Example 1 — extract the SKU family and count by it

```
index=web sourcetype=access_combined
| rex field=productId "(?<sku_family>[A-Z]+)-"
| stats count BY sku_family | sort - count
```

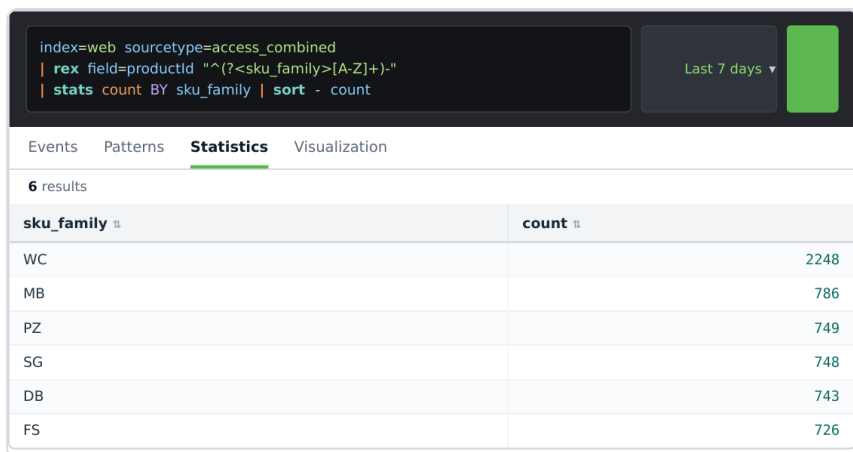


Fig 4.1 — `rex` pulls the letters before the first dash into `sku_family`; WC (shooters/tees) leads volume.

Example 2 — extract several fields from `_raw` at once

```
index=security sourcetype=linux_secure
| rex "for (?:invalid user )?(?<auth_user>\S+) from (?<attacker_ip>\d+\.\d+\.\d+\.\d+) port (?<src_port>\d+)"
| stats count BY auth_user, attacker_ip
```

Example 3 — sed mode to mask the session cookie

```
index=web
| rex field=JSESSIONID mode=sed "s/\w(?:=\w{4})/X/g"
```

This replaces all but the last four characters of the session id with `X` — a common way to keep a value linkable while removing its sensitivity.

MAX_MATCH FOR MULTIVALUE

By default `rex` captures the first match. Set `max_match=0` to capture *all* occurrences into a multivalue field — perfect for pulling every `productId` out of a multi-item request line.

ORDER IN THE PIPELINE

`rex` runs where you place it. Put it *before* the `stats` / `where` that depends on its fields, and after any filtering that reduces the event count it must scan.

05 · Regex Best Practices

A regex that works on ten events can cripple a search over ten million. These practices keep extractions correct, fast, and readable.

Practice	Why it matters
Anchor to nearby literals	Matching <code>productId=</code> first shrinks the search space and prevents accidental matches elsewhere in the event.
Prefer specific classes over <code>.</code>	<code>[^&]+</code> or <code>\d+</code> tells the engine exactly where to stop; <code>.*</code> forces expensive backtracking.
Use lazy quantifiers deliberately	<code>.*?</code> stops at the first delimiter instead of the last — usually what you want between two markers.
Use non-capturing groups	<code>(?:...)</code> for grouping you don't need to keep avoids polluting your field set and is marginally faster.
Name every capture	Positional groups are fragile; <code>(?<name>...)</code> is self-documenting and order-independent.
Avoid catastrophic backtracking	Nested quantifiers like <code>(a+)+</code> can hang the engine. Keep alternations and repetition flat.
Filter first, extract later	Reduce the event set with keyword/field filters <i>before</i> <code>rex</code> so the pattern runs on fewer events.
Promote stable extractions to config	Move proven patterns into <code>props.conf</code> / the Field Extractor so they apply automatically and are reused, not retyped.

READABLE IS MAINTAINABLE

Splunk supports commented, whitespace-friendly patterns with the `(?x)` flag. For a gnarly extraction, break it across lines with comments — future-you will thank present-you.

SEARCH-TIME VS INDEX-TIME

Keep extractions at search time by default — they're flexible and cost nothing to change. Only push to indexed extractions for fields you filter on constantly at massive scale, and only after measuring the win.

PART B

Self-Describing Data & Files

JSON and XML already carry their own field names — you shouldn't need a regex to read them. **spath** walks the structure and hands you dotted paths; the **spath()** eval function pulls a single value inline; and **multikv** turns a table printed inside one event into many tidy rows.

06 • The spath Command — JSON & XML

`spath` ("search path") extracts fields from structured `_raw` — JSON or XML — using location paths. Run it with no arguments and it auto-discovers every leaf; give it a path and it extracts just that one. Nested objects become **dot paths** (`geo.country`) and arrays become **curly paths** (`cart.items{}.productId`).

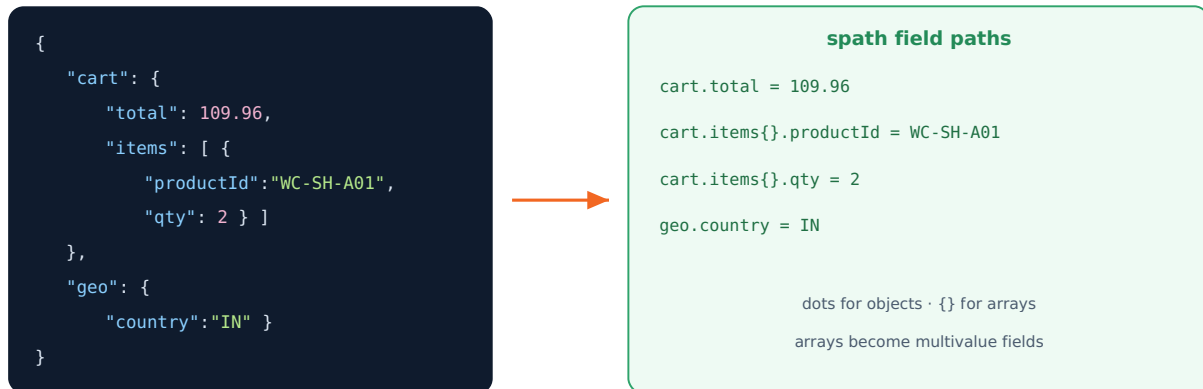


Fig 6.1 — `spath` maps a JSON structure to dotted/curly field paths.

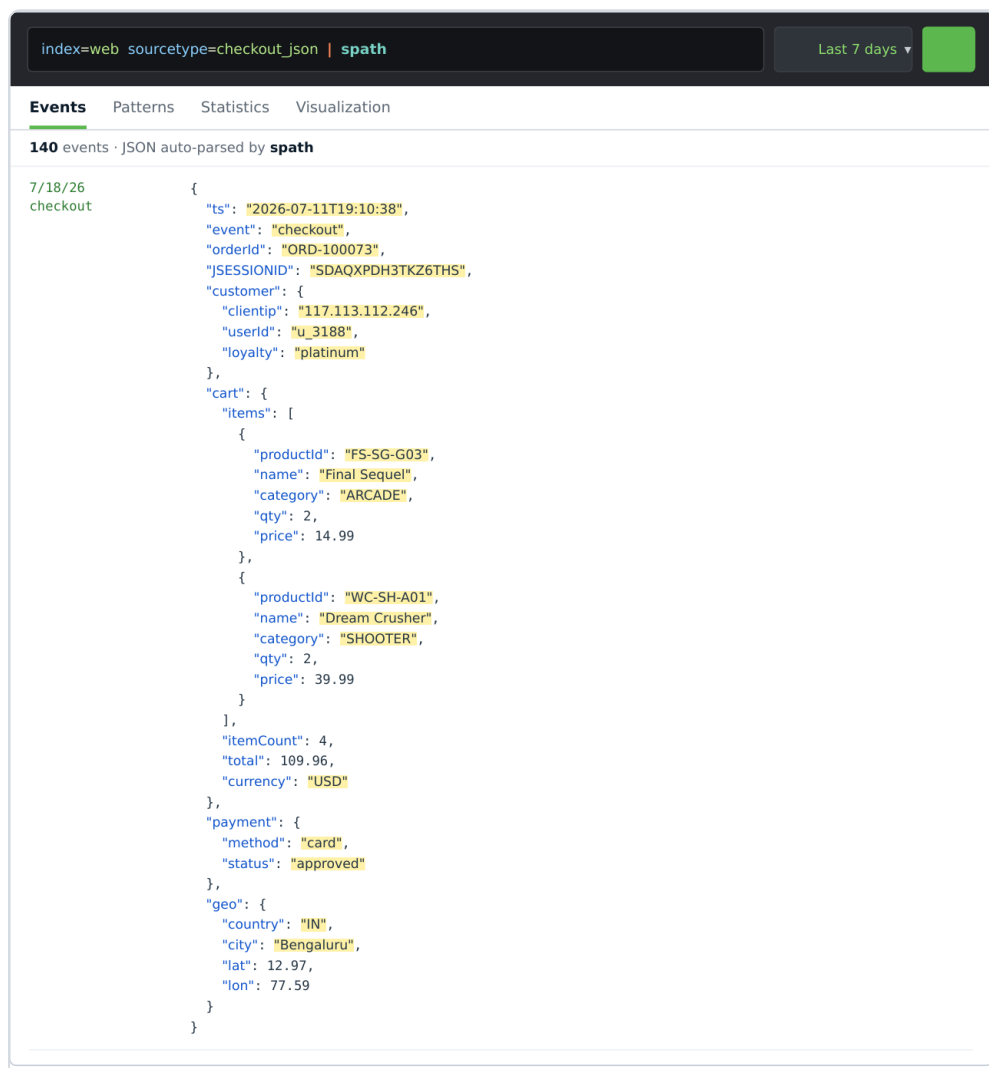


Fig 6.2 — A `checkout_json` event; `spath` auto-extracts every leaf into a field.

Example — revenue by country from nested JSON

```
index=web sourcetype=checkout_json
| spath
| stats count, sum(cart.total) AS revenue BY geo.country | sort - revenue
```

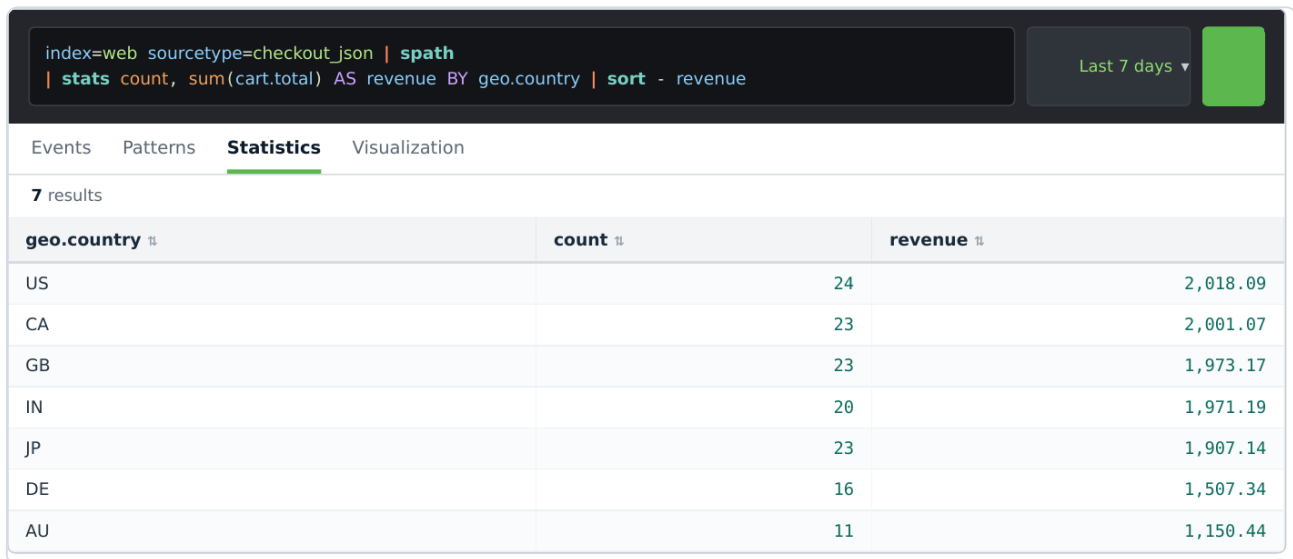


Fig 6.3 — Aggregating a JSON field (`cart.total`) grouped by a nested field (`geo.country`).

Working with arrays

```
# explode the items array so each product becomes its own row
index=web sourcetype=checkout_json | spath path=cart.items{}.productId output=pid
| mvexpand pid | stats count BY pid
```

XML TOO

The same command reads XML — `| spath output=user path=response.user.name`. For XML attributes use `@attr` in the path.

AUTO-KV VS SPATH

If the sourcetype is configured with `KV_MODE=json`, Splunk already exposes the paths and you may not need to call `spath` at all. Calling it explicitly is always safe and makes intent obvious.

07 · eval with the spath() Function

The `spath()` eval function extracts a single value from a JSON/XML string **inside an eval expression**. Use it when the structured data lives in a *field* (not `_raw`), when you need exactly one value without materialising every leaf, or when you want to combine the extraction with other eval logic in one step.

Syntax

```
eval out = spath(<input_field>, "<path>")
```

Example 1 — pull one nested value from a JSON field

```
index=web sourcetype=checkout_json
| eval pay_status = spath(_raw, "payment.status"),
      country     = spath(_raw, "geo.country")
| stats count BY country, pay_status
```

Example 2 — extract, then compute, in one pass

```
index=web sourcetype=checkout_json
| eval total    = tonumber(spath(_raw, "cart.total")),
      loyalty   = spath(_raw, "customer.loyalty"),
      tier       = if(total >= 80, "large_basket", "standard")
| stats avg(total) AS avg_basket BY loyalty, tier
```

SPATH COMMAND VS SPATH() FUNCTION

The

COMMAND

extracts many fields from `_raw` across the whole event; the

FUNCTION

extracts one value from any field into a variable you name. Reach for the function when the JSON is embedded in a field, or when you want the value only as an intermediate in a bigger eval.

GUARD AGAINST NULLS

Wrap in `coalesce(spath(...), "n/a")` when a path may be absent, so downstream `stats` don't silently drop rows.

08 • The multikv Command — Table-Formatted Events

Some sources log a whole table inside a single event — the output of `ps`, `netstat`, `df`, a scheduler report. `multikv` reads the column header row and turns each data row into its own event, with one field per column. One tabular event in, many structured events out.

How it reads a table

Option	Effect
<code>forceheader=<n></code>	Treat line <i>n</i> as the header when Splunk can't auto-detect it.
<code>fields <a> </code>	Keep only the named columns.
<code>filter <term></code>	Keep only rows containing a term — filter before the rows even become events.
<code>multitable=<bool></code>	Handle several tables in one event.

Example — explode a process report into per-process rows

```
index=web sourcetype=proc_report
| multikv forceheader=1
| table PID USER CPU MEM COMMAND
```

The screenshot shows a Splunk search interface. The search bar contains the query: `index=web sourcetype=proc_report | multikv forceheader=1 | table PID USER CPU MEM COMMAND`. The results are displayed in a table format under the 'Statistics' tab. The table has five columns: PID, USER, CPU, MEM, and COMMAND. It contains five rows of data, each representing a process extracted from a single tabular event.

PID	USER	CPU	MEM	COMMAND
5701	postgres	27.5	33.7	node
7697	root	29.1	35.5	nginx
1018	www-data	68.3	3.5	kube-apiserver
3075	splunk	78.8	10.9	sshd
306	root	85.8	30.2	sshd

Fig 8.1 — Five rows of a single fixed-width event become five events with typed columns.

Then treat the columns like any field

```
index=web sourcetype=proc_report | multikv forceheader=1
| stats avg(CPU) AS avg_cpu, max(MEM) AS peak_mem BY COMMAND | sort - avg_cpu
```

FIXED-WIDTH FRIEND

`multikv` is built for aligned columns. For delimited text (CSV-in-a-field) prefer `| extract` with a delimiter or a quick `rex` — right tool, right shape.

PART C

Field Aliases & Calculated Fields

Extraction gets a field to exist; **aliases** and **calculated fields** make it consistent and reusable. Define a naming convention once, alias divergent source fields onto it, and let calculated fields compute derived values automatically — so every analyst searches the same vocabulary without retyping eval.

09 • Field Naming Conventions

Field names are an API that every dashboard, alert, and analyst depends on. A convention decided up front prevents the sprawl of `srcip`, `src_ip`, `source_ip` and `clientip` all meaning the same thing.

Guideline	Rationale
Lowercase, snake_case	Fields are case-sensitive; one style avoids <code>Status</code> vs <code>status</code> misses.
Align to a data model / CIM	Splunk's Common Information Model defines standard names (<code>src</code> , <code>dest</code> , <code>user</code> , <code>action</code>) that unlock prebuilt content and cross-source correlation.
Name the meaning, not the source	<code>src_ip</code> beats <code>f3</code> or <code>col7</code> ; it survives log-format changes.
Reserve _-prefixed names	Leading-underscore fields (<code>_time</code> , <code>_raw</code>) are internal — don't create your own.
Be consistent with units	Encode units when ambiguous: <code>size_bytes</code> , <code>duration_ms</code> .

CIM AS THE TARGET

When you alias and calculate toward CIM names, Splunk Enterprise Security and prebuilt apps work against *your* data with zero rework. Treat CIM as the canonical vocabulary the rest of this part maps onto.

10 · Creating & Using Field Aliases

A **field alias** is a second name for an existing field. It doesn't copy data or cost extraction — it simply lets the same values answer to a canonical name across sources. Our web logs call the visitor `clientip`; our auth logs call the origin `src_ip`. Alias both to `src` and a single search spans them.

Define it (Settings → Fields → Field aliases)

Setting	Value (example)
Name	alias_client_to_src
Apply to · sourcetype	access_combined
Field alias	clientip AS src

The equivalent `props.conf` stanza:

```
[access_combined]
FIELDALIAS-client = clientip AS src
```

Now one search unifies both indexes

```
# clientip (web) and src_ip (security) both alias to src
(index=web OR index=security)
| stats count BY src | sort - count
```

ALIAS, DON'T RENAME

The original field still works — an alias *adds* a name, it doesn't remove one. Aliases are evaluated after extraction and before calculated fields and lookups, so later stages can rely on the canonical name.

ONE DIRECTION, SAME VALUES

An alias points a new name at existing values. If two sources genuinely differ (e.g. one stores a hostname, the other an IP), normalise them with a *calculated field* instead — see next chapter.

11 · Creating & Using Calculated Fields

A **calculated field** is a saved `eval` that Splunk applies automatically at search time — as if the field had been in the data all along. Define `line_total = qty * price` once, and every search, dashboard and alert sees `line_total` without anyone writing the eval again.

Define it (Settings → Fields → Calculated fields)

Setting	Value (example)
Apply to · sourcetype	access_combined
Field name	revenue
Eval expression	if(action="purchase", price, 0)

The equivalent `props.conf` stanza:

```
[access_combined]
EVAL-revenue = if(action="purchase", price, 0)
EVAL-status_family = case(status<300,"ok", status<400,"redirect", status<500,"client_err",
true(),"server_err")
```

Use it as if it were native

```
index=web | stats sum(revenue) AS total_revenue BY status_family
```

No `eval` in the search — `revenue` and `status_family` simply exist because the calculated fields compute them on the fly.

DRY ANALYTICS

Any eval you find yourself pasting into search after search is a calculated field waiting to be born. Define it centrally and every consumer stays consistent.

ORDER OF OPERATIONS

Search-time fields resolve in a fixed order: extractions → field aliases → calculated fields → lookups. So a calculated field can reference an alias, and a lookup can key off a calculated field — but not the reverse.

PART D

Exploring Lookups

Lookups bring in context that never appeared in the log — a product's name, a status code's meaning, an IP's threat reputation, a city's coordinates, a customer record from a database. Match on a key, enrich with new fields, or filter events by presence in a list. Five backends, one mental model.

12 · Including & Excluding Events with Lookups

The core operation is simple: `lookup` takes the value of a key field in each event, finds the matching row in a table, and adds that row's output fields to the event. Enrichment is the obvious use — but the same mechanism, combined with a following `search`, becomes a powerful **allow-list** / **deny-list filter**.

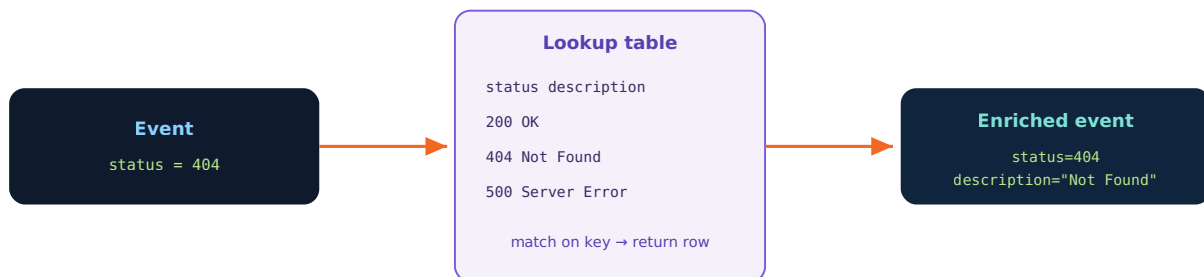


Fig 12.1 — A lookup matches on a key and appends the matching row's fields.

Enrich — add meaning to raw codes

```

index=web sourcetype=access_combined
| lookup http_status_lookup status OUTPUT status_description, status_type
| stats count BY status_type | sort - count
  
```

status_type	count
Success	4,921
Redirect	705
Client Error	197
Server Error	177

Fig 12.2 — Numeric status codes enriched into human-readable families via a CSV lookup.

Include — keep only events that match a list

```

# surface only failed logins from known-bad source IPs
index=security vendor_action="Failed password"
| lookup threat_ip_lookup src_ip OUTPUT threat_type, severity
| search threat_type=*
| stats count BY src_ip, threat_type, severity
  
```

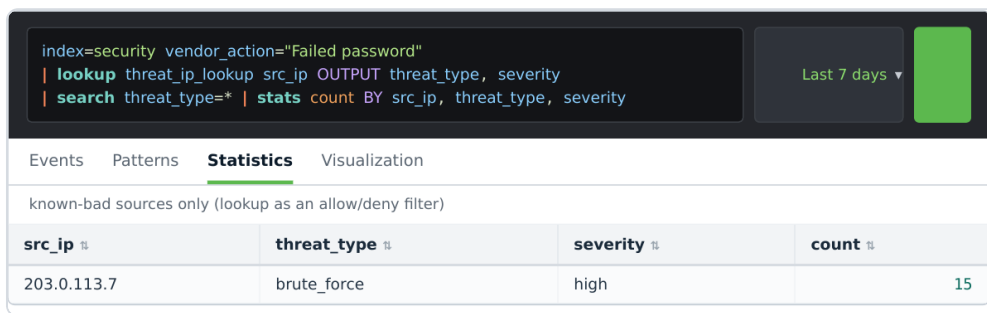


Fig 12.3 — Lookup-as-filter: only sources present in the threat list survive; 203.0.113.7 is caught.

Exclude — drop events that match a list

```
# everything EXCEPT known-good internal scanners
index=security
| lookup threat_ip_lookup src_ip OUTPUT threat_type
| search NOT threat_type=*
```

FILTER PATTERN

After a `lookup`, an output field is *present* only when the key matched. So `search field=*` keeps matches (allow-list) and `search NOT field=*` keeps non-matches (deny-list). No `inputlookup` subsearch required.

INPUTLOOKUP / OUTPUTLOOKUP

Read a table directly as events with `| inputlookup threat_ip_lookup`; write search results back into a lookup with `| outputlookup mytable.csv` — the basis for building and maintaining allow/deny lists from Splunk itself.

13 · KV Store Lookups

A CSV lookup is a flat file — great for static reference data, awkward when the data changes often or grows large. The **App Key Value Store** (KV Store) backs a lookup with a MongoDB collection instead: it supports per-record reads and writes, concurrent updates, larger volumes, and REST access. Use it for state that *changes* — asset inventories, enrichment that operators edit, running watchlists.

CSV vs KV Store

Dimension	CSV lookup	KV Store lookup
Backing	Flat file on disk	MongoDB collection
Writes	Rewrite whole file	Per-record insert/update/delete
Scale	Best for small/medium	Large, frequently updated
Access	File only	REST API, per-field typing, accelerated fields
Best for	Static reference (status codes)	Mutable state (asset owners, watchlists)

Define it

Declare the collection in `collections.conf`, then a lookup definition of type `kvstore` in `transforms.conf`:

```
[asset_owners]           # collections.conf – the collection

[asset_owners_lookup]    # transforms.conf – the lookup definition
external_type = kvstore
collection = asset_owners
fields_list = _key, host, owner, environment
```

Use it exactly like any lookup

```
index=security
| lookup asset_owners_lookup host OUTPUT owner, environment
| stats count BY environment, owner
```

Read & write records

```
# read the collection as events
| inputlookup asset_owners_lookup
# upsert the current host inventory back into it
index=security | stats values(vendor_action) BY host | outputlookup asset_owners_lookup
append=true
```

ACCELERATED FIELDS

Add accelerations on frequently-matched KV Store fields for lookup performance at scale — the KV Store can index them, something a CSV can't do.

14 · External (Scripted) Lookups

When the enrichment can't be a static table — it needs a live API call, a computation, or a query to a system Splunk doesn't natively speak to — an **external lookup** runs a script (typically Python). Splunk streams the input fields to the script's stdin as CSV and reads the enriched CSV back from stdout.

The classic: dnslookup

Splunk ships `external_lookup` for DNS both ways:

```
# resolve source IPs to hostnames on the fly
index=security vendor_action="Failed password"
| lookup dnslookup clientip AS src_ip OUTPUT clienthost AS src_host
```

Define your own

```
[my_api_lookup]                                # transforms.conf
external_cmd = my_enrich.py src_ip reputation
fields_list = src_ip, reputation
python.version = python3
```

The script reads a CSV header with `src_ip`, does its work (call a threat-intel API, say), and emits the same rows with a `reputation` column filled in.

COST & SAFETY

External lookups run a process per lookup invocation and can call the network — they're the slowest option and a supportability risk. Cache results (write into a KV Store), bound the input event count, and handle timeouts in the script. Never use one where a CSV/KV Store table would do.

WHERE SCRIPTS LIVE

Place the executable in the app's `bin/` directory; grant it the right permissions. Splunk invokes it relative to the app, so external lookups ship and version with the app that owns them.

15 · Geospatial Lookups

A **geospatial lookup** matches coordinates to *polygons* defined in a KMZ/KML file, returning the region a point falls inside — country, state, sales territory, custom zone. It's what turns a stream of lat/long into a choropleth on the [Choropleth Map](#) visualization.

The flow

1. Get coordinates onto the event — from the data (our `geo.lat` / `geo.lon`) or via `iplocation`.
2. Register a KMZ of polygons as a lookup of type `geo`.
3. `lookup` the coordinates to get the enclosing feature (e.g. `featureId` = country/state).
4. Count by feature and render as a choropleth.

Example — sales by geographic region

```
index=web sourcetype=checkout_json | spath
| lookup geo_countries latitude AS geo.lat longitude AS geo.lon OUTPUT featureId
| stats sum(cart.total) AS revenue BY featureId
| geom geo_countries featureIdField=featureId
```

The trailing `geom` command attaches the polygon geometry so the Choropleth visualization can shade each country by revenue. Splunk ships `geo_us_states` and `geo_countries` KMZs out of the box.

NO COORDINATES? USE IPLOCATION FIRST

`| iplocation clientip` adds `lat`, `lon`, `City`, `Country` from the bundled MaxMind database — a fast way to geolocate the Day-1 access logs before a geospatial lookup.

CLUSTER ON THE MAP

For point maps (not regions), `geostats` aggregates by lat/long into map clusters directly — no KMZ needed. Geospatial lookups are specifically for *region* membership.

16 · Database Lookups (DB Connect)

When the reference data lives in a relational database — a product master, a CMDB, a customer table — **Splunk DB Connect** exposes it as a lookup without exporting to CSV. You define a database connection and an **identity** (credentials), then a DB-backed lookup that runs SQL against the source at search time.

Set-up path (DB Connect app)

Step	What you configure
1 · Driver	Install the JDBC driver for your DB (MySQL, Postgres, Oracle, MSSQL...).
2 · Identity	Store the DB username/password DB Connect will authenticate with.
3 · Connection	Host, port, database, and the identity to use.
4 · DB Lookup	Map an event field to a table column and choose the columns to output.

Use it like any lookup

```
# enrich each SKU with live catalogue data from the product database
index=web action=purchase
| lookup product_db_lookup productId OUTPUT product_name, supplier, margin
| stats sum(price) AS revenue, avg(margin) AS avg_margin BY supplier
```

Alternatively, `| dbxquery` runs arbitrary SQL and streams rows in as events — useful for pulling a whole reference table on a schedule into a KV Store cache.

LATENCY & LOAD

A DB lookup queries the source database while the search runs. For hot paths, cache the table into a KV Store (scheduled `dbxquery` → `outputlookup`) and look up against the cache, so a heavy Splunk search never hammers production OLTP.

17 • Lookup Best Practices

Lookups are cheap to add and easy to misuse. These habits keep them fast, correct, and quiet.

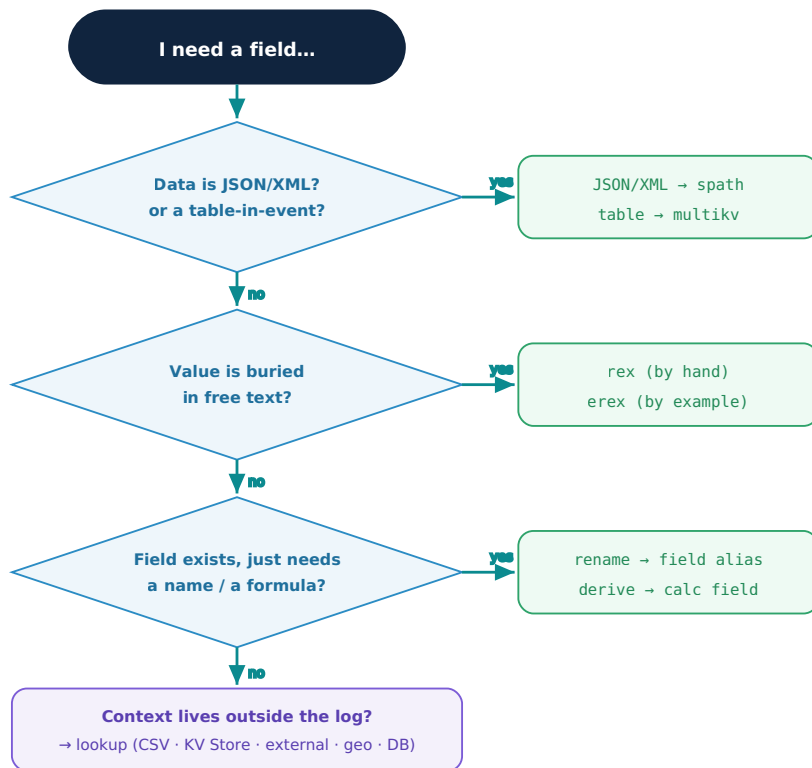
Practice	Why
Pick the right backend	Static & small → CSV. Mutable & large → KV Store. Live/computed → external. Relational source → DB Connect.
Match on a clean key	Normalise the key (case, trim, type) with an alias/calculated field <i>before</i> the lookup so matches don't silently miss.
Output only what you need	Naming specific <code>OUTPUT</code> fields avoids clobbering existing fields and keeps events lean. Use <code>OUTPUTNEW</code> to fill only when absent.
Enable automatic lookups sparingly	An automatic lookup runs on <i>every</i> search of that sourcetype — powerful, but a slow lookup taxes everyone. Reserve for high-value, fast tables.
Cache expensive sources	Snapshot external/DB data into a KV Store on a schedule; search against the cache.
Filter before you look up	Reduce the event set first — a lookup runs per event that reaches it.
Handle misses deliberately	Decide what an unmatched key means: default value, drop, or flag. Don't let nulls surprise downstream stats.
Version & permission tables	Ship lookups inside an app, set sharing/permissions explicitly, and keep them in source control alongside the searches that use them.

OUTPUT VS OUTPUTNEW

`OUTPUT` overwrites the named fields every time; `OUTPUTNEW` writes only when the field is currently null — the safe choice when enriching data that may already carry the field from another source.

18 · Appendix A — Extraction & Lookup Decision Flowchart

Which tool for the field in front of you? Follow the shape of the data.



Order at search time: extractions → aliases → calculated fields → lookups

Fig A.1 — Match the technique to the shape of the data.

19 · Appendix B — Day-2 Quick-Reference Cheat Sheet

EXTRACTION

```
# named capture with rex
| rex field=productId "^(?<fam>[A-Z]+)-"
# all matches → multivalued
| rex max_match=0 "id=(?<ids>\d+)"
# mask with sed mode
| rex field=JSESSIONID mode=sed "s/\w(?=\w{4})/X/g"
# learn a regex from examples
| erex token examples="SD1..., SDT8..."
```

STRUCTURED DATA

```
| spath                                # auto all
paths
| spath path=cart.items{}.productId
| eval c=spath(_raw,"geo.country")
| multikv forceheader=1
```

ALIASES & CALC FIELDS (PROPS.CONF)

```
[access_combined]
FIELDALIAS-c = clientip AS src
EVAL-revenue = if(action="purchase",price,0)
```

LOOKUPS

```
# enrich
| lookup http_status_lookup status OUTPUT
status_type
# fill only if absent
| lookup t key OUTPUTNEW f
# allow-list / deny-list
| search threat_type=*
| search NOT threat_type=*
# read / write tables
| inputlookup t | outputlookup t
# geo & ip
| iplocation clientip
| geom geo_countries featureIdField=f
```

BACKEND PICKER

Need	Use
Static, small	CSV
Mutable, large	KV Store
Live / computed	External
Region from lat/lon	Geospatial
Relational source	DB Connect

END OF DAY 2

You can now create fields from anything — raw text, JSON, tables — normalise them with aliases and calculated fields, and enrich or filter with every class of lookup. All examples run against the Buttercup Games dataset shipped in [Buttercup_Games_Dataset.zip](#).